Software Engineering

Course 9 – 12 May

Outline

- Previous courses…
  - Design Patterns (Creational Patterns, Structural Patterns, Behavioral Patterns)
- Behavioral Patterns
  - State
  - Strategy
  - Template Method
  - Visitor
- Other Design Patterns
- Quality Assurance

Previuos Courses

- GOF: Creational Patterns, Structural Patterns, Behavioral Patterns
- Creational Patterns
- Structural Patterns
- Behavioral Patterns

Previous Courses – CP

- **Abstract Factory** – computer components
- **Builder** – children meal
- **Factory Method** – Hello <Mr/Ms>
- **Prototype** – Cell division
- **Singleton** – server log files

SP

- **Adapter** – socket-plug
- **Bridge** – drawing API
- **Composite** – employee hierarchy
- **Decorator** – Christmas tree
- **Façade** – store keeper
- **Flyweight** – FontData
- **Proxy** – ATM access

BP

- Command
- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer

Behavioral Patterns

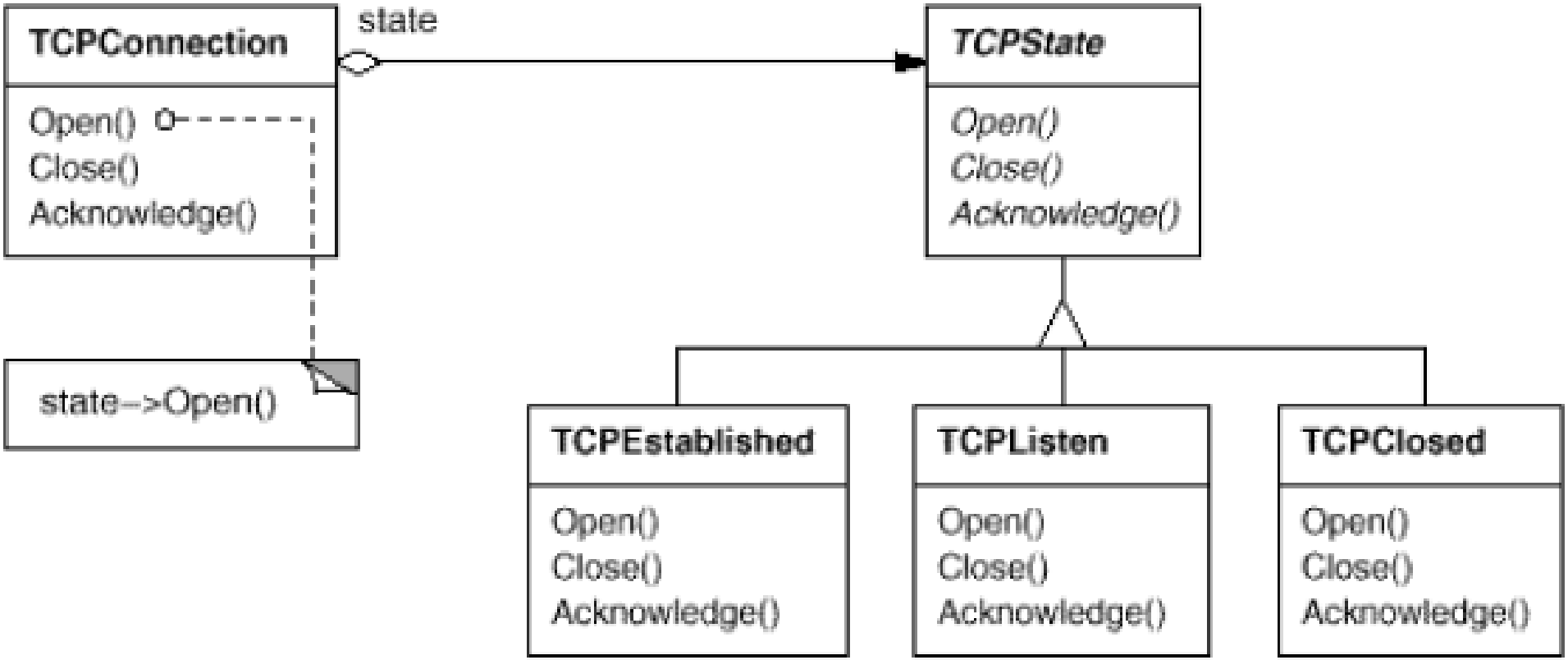
- State
- Strategy
- Template Method
- Visitor

State

- **Intent** – Allow an object to alter its behavior when its internal state changes
- **Also Known As** – Objects for States
- **Motivation** – Consider a class TCPConnection that represents a network connection. A TCPConnection object can be in one of several different states: Established, Listening, Closed. When a TCPConnection object receives requests from other objects, it responds differently depending on its current state

State – Idea

- The key idea in this pattern is to introduce an abstract class called TCPState to represent the states of the network connection.



State – Applicability

- Use the State pattern in either of the following cases:
  - An object's behavior depends on its state
  - Operations have large, multipart conditional statements that depend on the object's state

State – The Good, The Bad …

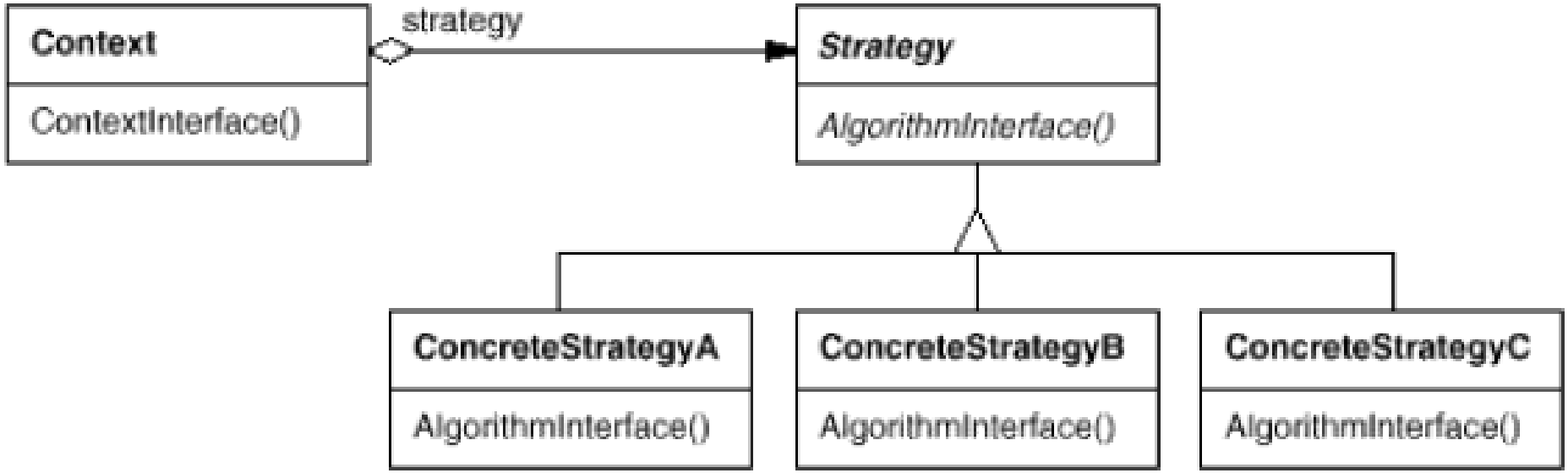
- Simplifies the code of the objects by removing lots of if statements
- Preserves S and O (from SOLID)
- Leads to large and unnecessary overhead for objects with few states or whose states rarely change

Strategy

- **Intent** – Define a family of algorithms, encapsulate each one, and make them inter changeable
- **Also Known As** – Policy
- **Motivation** – Many algorithms exist for breaking a stream of text into lines. Hard-wiring all such algorithms into the classes that require them isn't desirable for several reasons

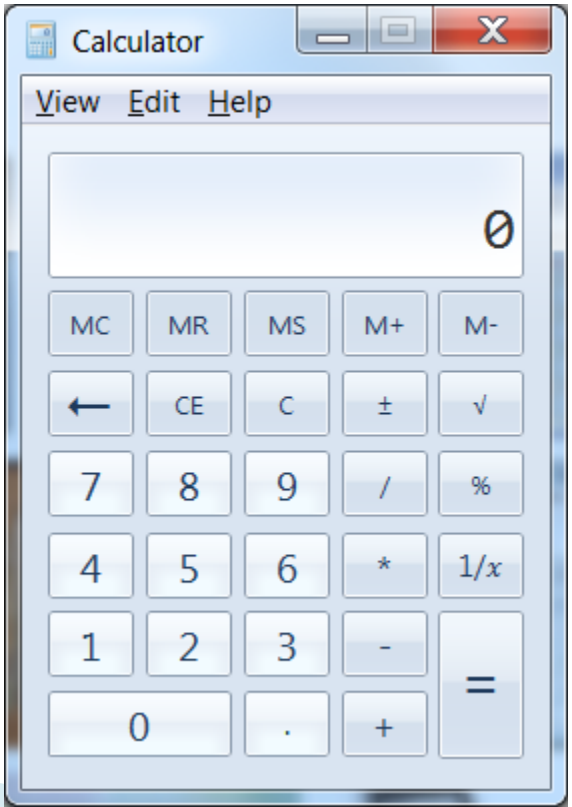
Strategy – Structure

- With Strategy pattern, we can define classes that encapsulate different line breaking algorithms



Strategy – Example

- In the strategy pattern algorithms can be selected at runtime.
- A standard calculator that implements basic operations: +, –, *



Strategy – Java 1

```
interface Strategy {
   int execute(int a, int b);
}
class ConcreteStrategyAdd implements Strategy {
   public int execute(int a, int b) {
         System.out.println("Called ConcreteStrategyA's execute()");
         return (a + b);
}}
class ConcreteStrategySub implements Strategy {
   public int execute(int a, int b) {
         System.out.println("Called ConcreteStrategyB's execute()");
         return (a - b);
}}
class ConcreteStrategyMul implements Strategy {
   public int execute(int a, int b) {
         System.out.println("Called ConcreteStrategyC's execute()");
         return a * b;
} }
```

Strategy – Java 2

```
class Context {
  Strategy strategy;
  public Context(Strategy strategy) {
      this.strategy = strategy;
  }
  public int execute(int a, int b) {
      return this.strategy.execute(a, b);
  }
}
```

Strategy – Java 3

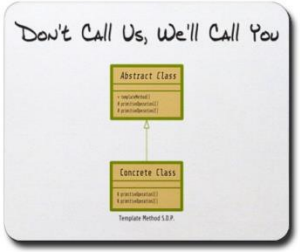
```
class StrategyExample {

  public static void main(String[] args) {
   Context context;
   context = new Context(new ConcreteStrategyAdd());
   int resultA = context.execute(3,4);
   context = new Context(new ConcreteStrategySub());
   int resultB = context.execute(3,4);
   context = new Context(new ConcreteStrategyMul());
   int resultC = context.execute(3,4);
  }
}
```

Strategy – The Good, The Bad …

- Can swap algorithms at runtime
- Replaces inheritance with composition
- Isolates the implementation of algorithm from the classes using those algorithm
- If you only need few algorithms, the extra complication of code is not useful
- Some functional languages can achieve the same effect by using anonymous functions, and require less code
- Users need to understand the differences between implementations to use them properly

Template Method



- **Intent – Define the skeleton of an algorithm** in an operation, deferring some steps to subclasses
- **Motivation –** Consider an application framework that provides Application and Document classes.
- The Application class is responsible for opening existing documents stored in an external format, such as a file. A Document object represents the information in a document once it's read from the file

Template Method – Example

- The template pattern is often referred to as the **Hollywood Principle**: "*Don't call us, we'll call you*." Using this principle, the template method in a parent class controls the overall process by calling subclass methods as required

- This is shown in several games in which players play against the others, but only one is playing at a given time

Template Method – Java 1

```
abstract class Game {
  protected int playersCount;
  abstract void initializeGame();
  abstract void makePlay(int player);
  abstract boolean endOfGame();
  abstract void printWinner();

  final void playOneGame(int playersCount) {
      this.playersCount = playersCount;
      initializeGame(); int j = 0;
      while (!endOfGame()) {
          makePlay(j); j = (j + 1) % playersCount; }
       printWinner();
  }
}
```

Template Method – Java 2

```
class Monopoly extends Game {
  // Implementation of necessary concrete methods

  void initializeGame() { // ... }
  void makePlay(int player) { // ... }
  boolean endOfGame() { // ... }
  void printWinner() { // ... }

  // Specific declarations for the Monopoly game.
}

class Chess extends Game {
...}
```

Template Method – The Good, The Bad …

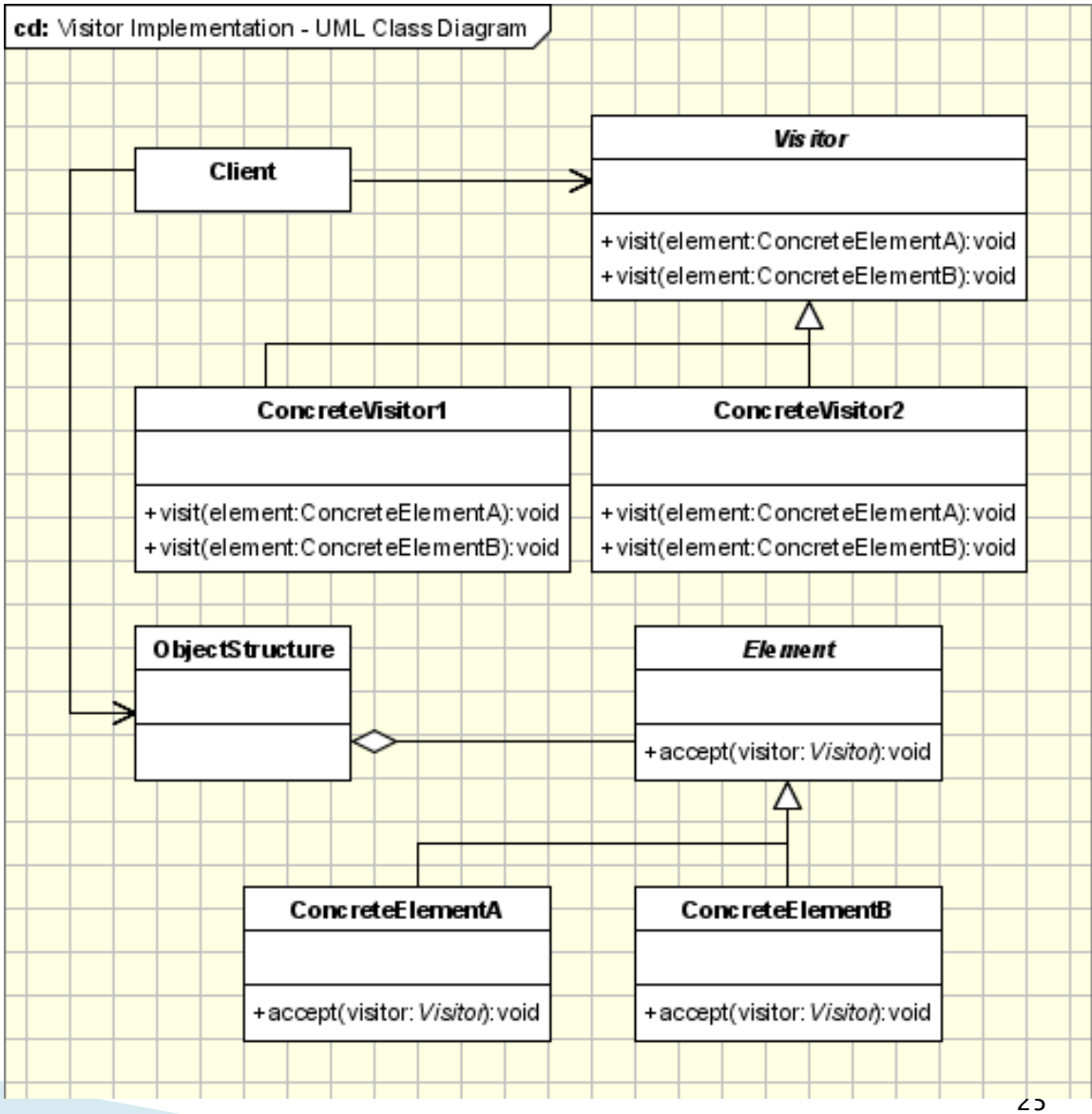
- Clients can choose to override only some parts of the algorithm, and are less affected by changes to other segments
- Duplicate code can be sent to a superclass
- If you override some part of the algorithm, it can lead to breaking Liskov substitution
- The skeleton of the algorithm may not suit some clients
- The more elements in the template, the more difficult it is to manage

Visitor

- **Intent** – Represent an operation to be performed on the elements of an object structure. **Visitor lets you define a new operation without changing the classes** of the elements on which it operates.
- **Motivation** – Collections are data types widely used in object oriented programming. Often collections contain objects of different types and in those cases some operations have to be performed on all the collection elements without knowing the type

Visitor – Structure Example

- We want to create a reporting module in our application to make statistics about a group of customers
- The statistics should be very detailed so all the data related to the customer must be parsed
- All the entities involved in this hierarchy must accept a visitor so the CustomerGroup, Customer, Order and Item are visitable objects



Visitor– Applicability

- The visitor pattern is used when:
  - Similar operations have to be performed on objects of different types grouped in a structure
  - There are many distinct and unrelated operations needed to be performed
  - The object structure is not likely to be changed but is very probable to have new operations which have to be added

Visitor Pattern using Reflection

- Reflection can be used to overcome the main drawback of the visitor pattern
- When the standard implementation of visitor pattern is used the method to invoke is determined at runtime
- **Reflection is the mechanism used to determine the method to be called at compile-time**

Visitor – The Good, The Bad …

- A visitor can accumulate information about visited objects as it passes through the collection, allowing for more informed decisions for later visitations
- Preserves S and O (from SOLID)
- A visitor may not be able to access private or protected fields of visited objects
- Every time you add an extra class to the collection, all visitors must be updated

Other Types of DP 1

- **Concurrency Patterns** – deal with multi-threaded programming paradigm
  - **Single Threaded Execution** – Prevent concurrent calls to the method from resulting in concurrent executions of the method
  - **Scheduler** – Control the order in which threads are scheduled to execute single threaded code using an object that explicitly sequences waiting threads
  - **Producer-Consumer** – Coordinate the asynchronous production and consumption of information or objects

Other Types of DP 2

- **Testing Patterns 1**
  - **Black Box Testing** – Ensure that software satisfies requirements
  - **White Box Testing** – Design a suite of test cases to exhaustively test software by testing it in all meaningful situations
  - **Unit Testing** – Test individual classes
  - **Integration Testing** – Test individually developed classes together for the first time
  - **System Testing** – Test a program as a whole entity

Other Types of DP 3

- **Testing Patterns 2**
  - **Regression Testing** – Keep track of the outcomes of testing software with a suite of tests over time
  - **Acceptance Testing** – Is done to ensure that delivered software meets the needs of the customer or organization that the software was developed for
  - **Clean Room Testing** – People designing software should not discuss specifications or their implementation with people designing tests for the software

Other Types of DP 4

- **Distributed Architecture Patterns**
  - **Mobile Agent** – An object needs to access very large volume of remote data => move the object to the data
  - **Demilitarized Zone** – You don't want hackers to be able to gain access to servers
  - **Object Replication** – You need to improve the throughput or availability of a distributed computation

Other Classes of DP

- **Transaction patterns** – Ensure that a transaction will never have any unexpected or inconsistent outcome. Design and implement transactions correctly and with a minimum of effort
- **Distributed computing patterns**
- **Temporal patterns** – for distributed applications to function correctly, the clocks on the computers they run on must be synchronized. You may need to access pervious or future states of an object. The values of an object's attributes may change over time
- **Database patterns**

OOD principles for classes

- The **S**ingle Responsibility Principle
- The **O**pen Closed Principle
- The **L**iskov Substitution Principle
- The **I**nterface Segregation Principle
- The **D**ependency Inversion Principle

- SOLID

The Single Responsibility Principle

- *A class should have one, and only one, reason to change.*
- A responsibility to is “a reason for change.”
- Each responsibility is an axis of change.

```
interface Modem
  {
  public void dial(String pno);
  public void hangup();
  public void send(char c);
  public char recv();
  }
```

The Single Responsibility Principle

- Should these responsibilities be separated?
  - If the implementations for the communication and connection management change independently, separately
  - If the implementations only change together, do not separate
- Corollary: An axis of change is only an axis of change if the changes actually occur.

The Open Closed Principle

- *You should be able to extend a class' behavior without modifying it.*
- Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.
- The primary mechanisms behind the Open-Closed principle are abstraction and polymorphism.

The Liskov Substitution Principle

- *Derived classes must be substitutable for their base classes.*
- Makes applications more maintainable, reusable and robust
- If there is a function which does not conform to the LSP, then that function uses a reference to a base class, but must know about all the derivatives of that base class.
- Such a function violates the Open-Closed principle

The Liskov Substitution Principle

```
public class Rectangle
{
public void SetWidth(double w) {itsWidth=w;}
public void SetHeight(double h) {itsHeight=w;}
public double GetHeight() const {return itsHeight;}
public double GetWidth() const {return itsWidth;}
protected double itsWidth;
protected double itsHeight;
};
```

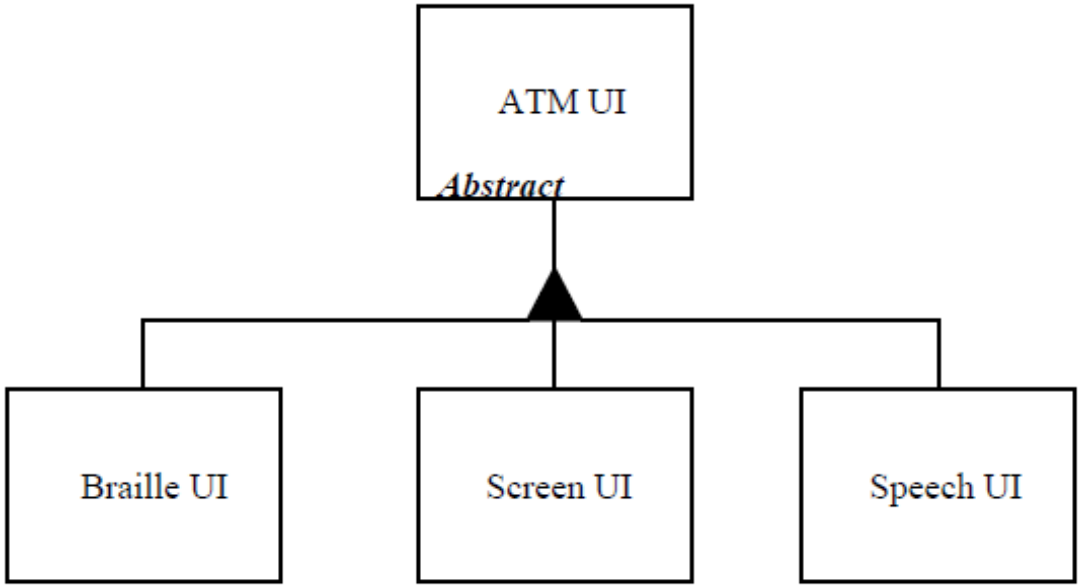
```
public class Square extends Rectangle
{
public void SetWidth(double w) {super.SetWidth(w);
                                super.SetHeight(w);}
public void SetHeight(double h) {super.SetWidth(h);
                                super.SetHeight(h);}
```

The Liskov Substitution Principle

```
public class Test
{
  public static void f(Rectangle r, float a)
  {
     r.SetWidth(a);
  }
}
public class Main
{
  public static void main(String args[])
  {
     Square square = new Square();
     square.SetWidth(10);
     Test test = new Test();
      test.f(square);
     //at this point the square is not a mathematical square
     //because its height is different from its length
  }
};
```

The Interface Segregation Principle

- *Clients should not be forced to depend upon interfaces that they do not use.*
- *Make fine grained interfaces that are client specific.*



The Dependency Inversion Principle

- What is it that makes a design bad?
  - most software eventually degrades to the point where someone will declare the design to be unsound
  - Because of the lack of a good working definition of “bad” design.

The Dependency Inversion Principle

- The Definition of a “Bad Design”
  - It is hard to change because every change affects too many other parts of the system. (Rigidity)
  - When you make a change, unexpected parts of the system break. (Fragility)
  - It is hard to reuse in another application because it cannot be separated from the current application. (Immobility)

The Dependency Inversion Principle

- A. High level modules should not depend upon low level modules. Both should depend upon abstractions.
- B. Abstractions should not depend upon details. Details should depend upon abstractions.

Object-Oriented Design

- The most common types of programming are Structured Programming and Object Oriented Programming
- It has become difficult to write a program that does not have the external appearance of both structured programming and object oriented programming
  - Do not have **goto**
  - class based and do not support functions or variables that are not within a class
- Programs may look structured and object oriented, but looks can be deceiving

Organization of Classes

- As software applications grow in size and complexity they require some kind of high level organization.
- The class is too finely grained to be used as an organizational unit for large applications.
- Something “larger” than a class is needed => packages.

OOD principles for deliverables

- Cohesion
  - The Release Reuse Equivalency Principle
  - The Common Closure Principle
  - The Common Reuse Principle

- Coupling
  - Acyclic Dependencies Principle
  - The Stable Dependencies Principle
  - The Stable Abstractions Principle

Designing with Packages

- What are the best partitioning criteria?
- What are the relationships that exist between packages, and what design principles govern their use?
- Should packages be designed before classes (Top down)? Or should classes be designed before packages (Bottom up)?
- How are packages physically represented? In the programming language? In the development environment?
- Once created, how will we use these packages?

The Reuse/Release Equivalence Principle

- Code copying vs. code reuse
- I reuse code if, and only if, I never need to look at the source code. The author is responsible for maintenance
  - I am the customer
  - When the libraries that I am reusing are changed by the author, I need to be notified
  - I may decide to use the old version of the library for a time
  - I will need the author to make regular releases of the library
  - I can reuse nothing that is not also released

The Reuse/Release Equivalence Principle

- The granule of reuse is the granule of release. Only components that are released through a tracking system can be effectively reused. This granule is the package.

The Common Reuse Principle

- *The classes in a package are reused together. If you reuse one of the classes in a package, you reuse them all.*
- Which classes should be placed into a package?
  - Classes that tend to be reused together belong in the same package.
- Packages to have physical representations that need to be distributed.

The Common Reuse Principle

- I want to make sure that when I depend upon a package, I depend upon every class in that package or I am wasting effort.

The Common Closure Principle

- The classes in a package should be closed together against the same kinds of changes. A change that affects a package affects all the classes in that package.
- If two classes are so tightly bound, either physically or conceptually, such that they almost always change together; then they belong in the same package.

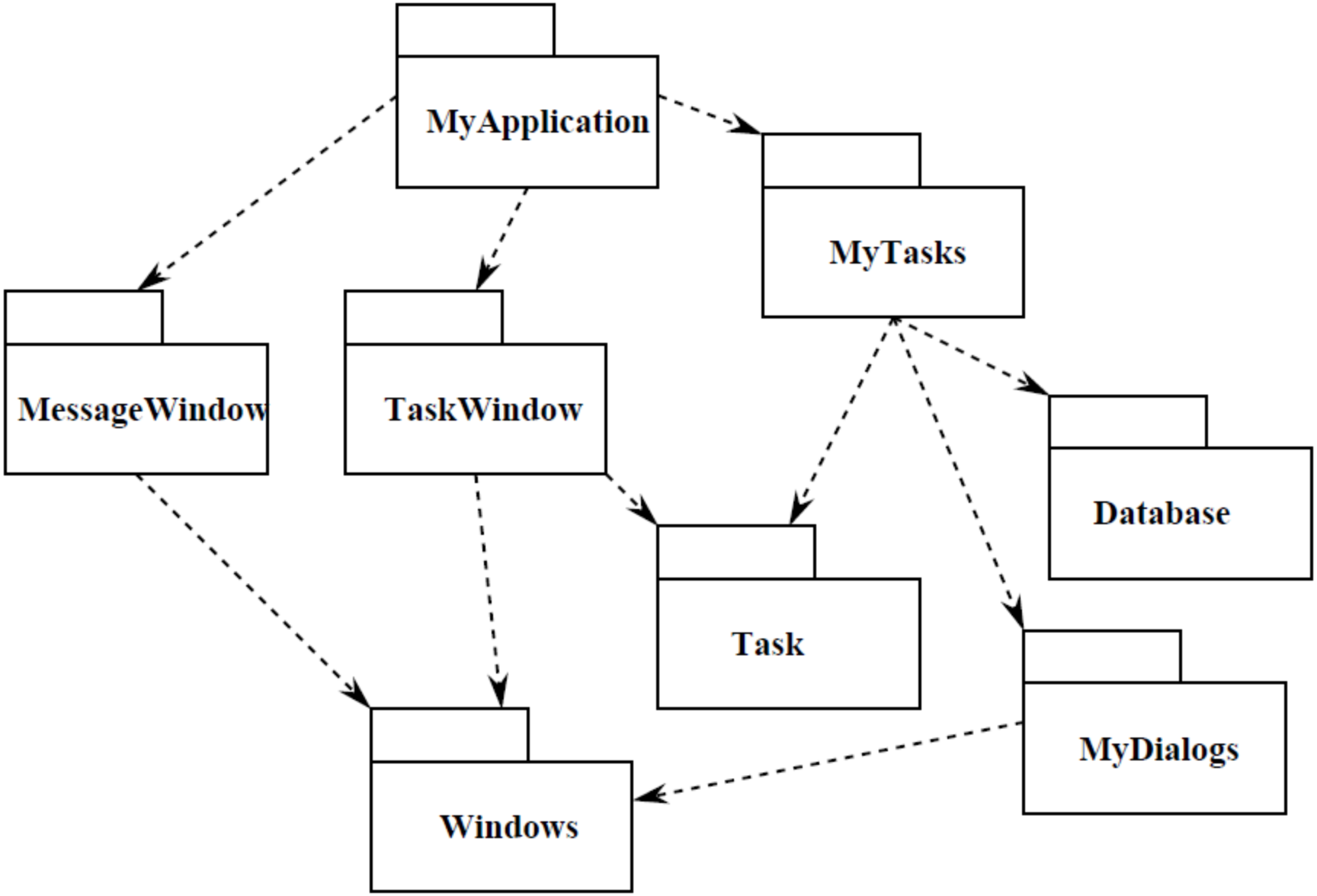
The Acyclic Dependencies Principle

- The morning after syndrome: you make stuff work and then gone home; next morning it longer works? Why? Because somebody **stayed later than you**!
- Many developers are modifying the same source files.
- Partition the development environment into releasable packages
- You must *manage the dependency structure of the packages*

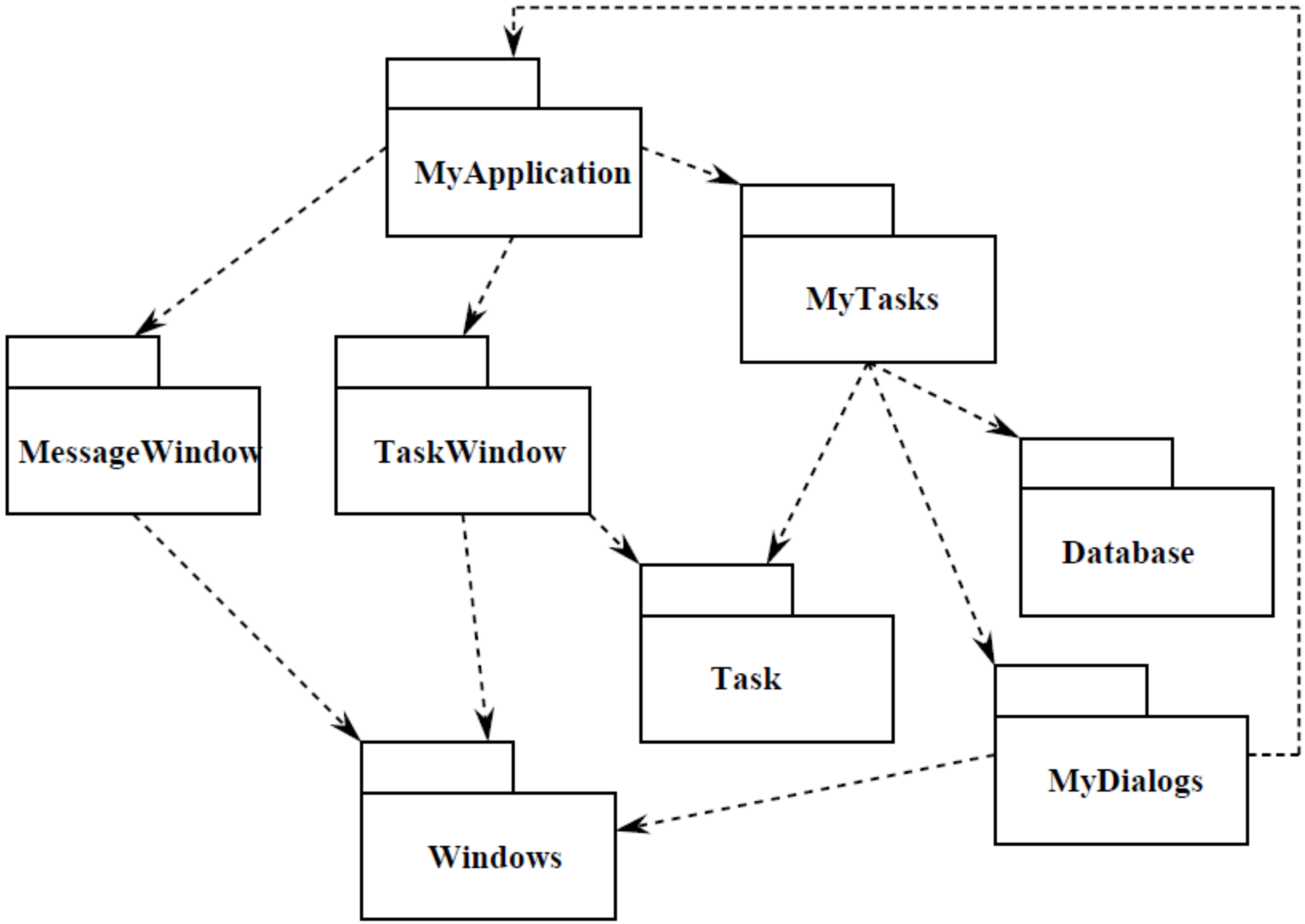
The Acyclic Dependencies Principle

- The dependency structure between packages must be a directed acyclic graph (DAG). That is, there must be no cycles in the dependency structure.

The Acyclic Dependencies Principle



The Acyclic Dependencies Principle



The Acyclic Dependencies Principle

- Breaking the Cycle
  - Apply the Dependency Inversion Principle (DIP). Create an abstract base class
  - Create a new package that both MyDialogs and MyApplication depend upon. Move the class(es) that they both depend upon into that new package.

- The package structure cannot be designed from the top down.

Stability

- Not easily moved
- A measure of the difficulty in changing a module
- Stability can be achieved through
  - Independence
  - Responsibility
- The most stable classes are Independent and Responsible. They have no reason to change, and lots of reasons not to change.

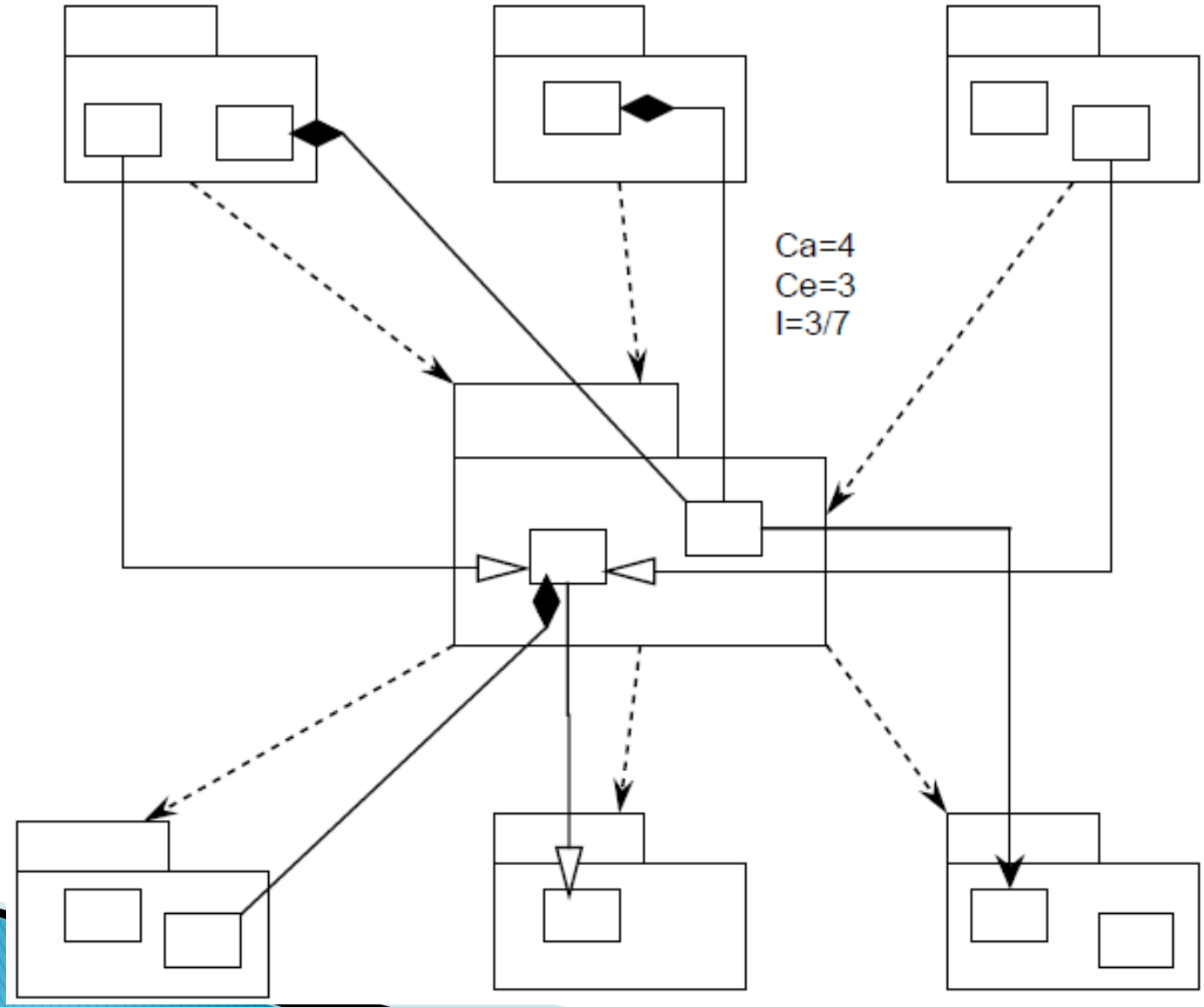
The Stable Dependencies Principle

- The dependencies between packages in a design should be in the direction of the stability of the packages. A package should only depend upon packages that are more stable that it is.
- We ensure that modules that are designed to be unstable are not depended upon by modules that are more stable

Stability Metrics

- **Ca** : Afferent Couplings : The number of classes outside this package that depend upon classes within this package.
- **Ce**: Efferent Couplings : The number of classes inside this package that depend upon classes outside this package.
- **I** : Instability : (Ce/(Ca+Ce)) I=0 maximally stable package. I=1 maximally instable package.

Stability Metrics



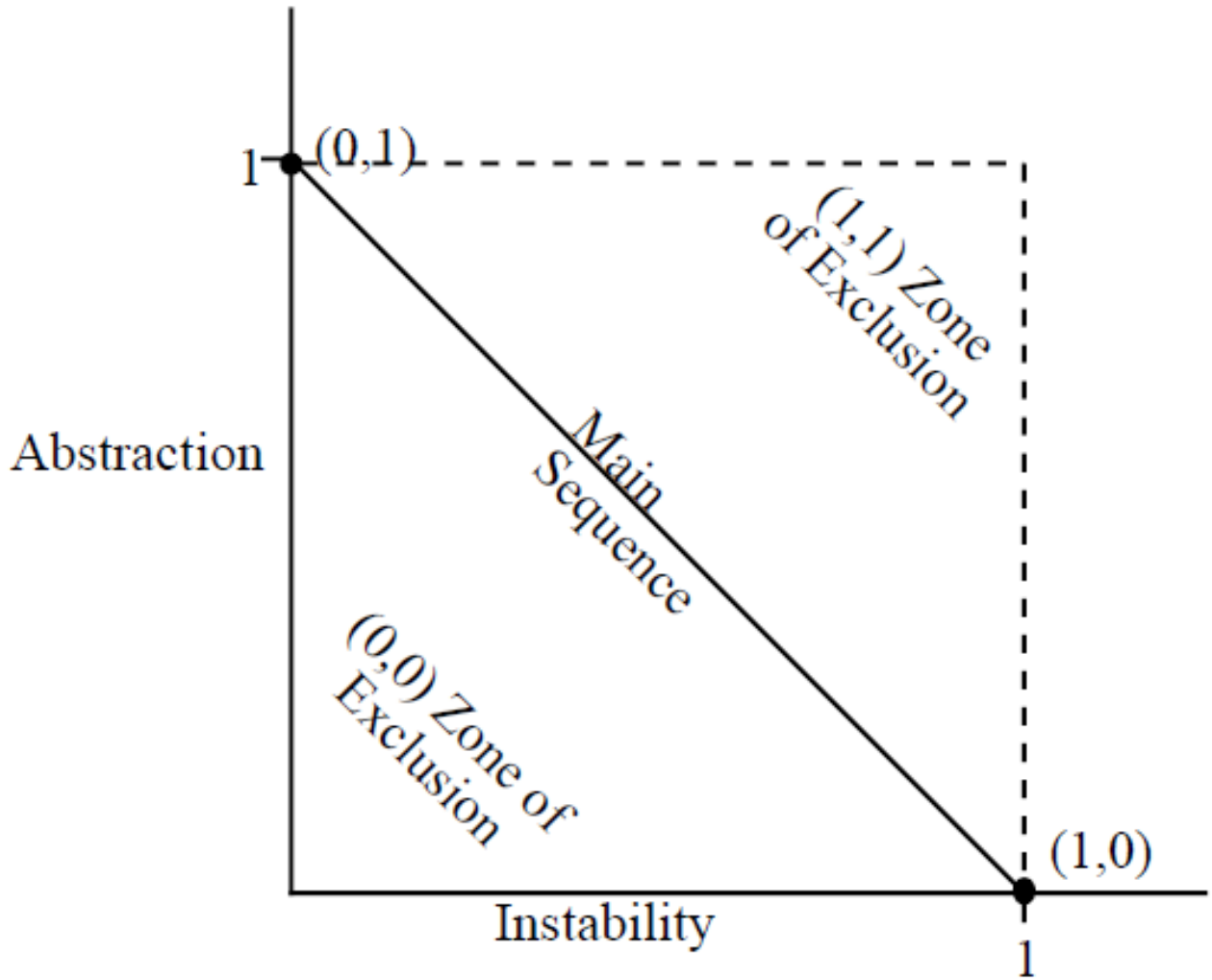
The Stable Dependencies Principle

- Not all packages should be stable
- The software the encapsulates the high level design model of the system should be placed into stable packages
- How can a package which is maximally stable (I=0) be flexible enough to withstand change?
  - classes that are flexible enough to be extended without requiring modification => abstract classes

The Stable Abstractions Principle

- Packages that are maximally stable should be maximally abstract. Instable packages should be concrete. The abstraction of a package should be in proportion to its stability.
- Abstraction (A) is the measure of abstractness in a package. A = AC/TC

Main Sequence



Design Patterns – Why?

- **If a problem occurs over and over again**, a solution to that problem has been used effectively (solution = pattern)

- When you make a design, you **should know the names of some common solutions**. Learning design patterns is good for people to **communicate each other effectively**

Design Patterns – Definitions

- "Design patterns capture solutions that have developed and evolved over time" (GOF – ***Gang-Of-Four*** (because of the four authors who wrote it), *Design Patterns: Elements of Reusable Object-Oriented Software)*

- **In software engineering (or computer science), a design pattern is a general repeatable solution to a commonly occurring problem in software design**

- The **design patterns** are language-independent strategies for solving common object-oriented design problems

How to Select a Design Pattern?

- With more than 20 design patterns to choose from, it might be hard to find the one that addresses a particular design problem

- Approaches to finding the design pattern that's right for your problem:
  1. *Consider how design patterns solve design problems*
  2. *Scan Intent sections*
  3. *Study relationships between patterns*
  4. *Study patterns of like purpose (comparison)*
  5. *Examine a cause of redesign*
  6. *Consider what should be variable in your design*

How to Use a Design Pattern?

1. *Read the pattern once through for an overview*
2. *Go back and study the Structure*, Participants, and Collaborations sections
3. *Look at the Sample Code section to see a concrete example*
4. *Choose names for pattern participants that are meaningful in the application context*
5. *Define the classes*
6. *Define application-specific names for operations in the pattern*
7. *Implement the operations to carry out the responsibilities and collaborations in the pattern*

Bibliography

- Robert C. Martin, Engineering Notebook columns for The C++ Report
- Design Principles and Design Patterns. Robert C. Martin.
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)
- Adrian Iftene, Advanced Software Engineering Techniques

Links

- Structural Patterns: http://www.oodesign.com/structural-patterns/
- Gang-Of-Four: http://c2.com/cgi/wiki?GangOfFour, http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf
- Design Patterns Book: http://c2.com/cgi/wiki?DesignPatternsBook
- About Design Patterns: http://www.javacamp.org/designPattern/
- Design Patterns – Java companion: http://www.patterndepot.com/put/8/JavaPatterns.htm
- Java Design patterns: http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- Overview of Design Patterns: http://www.mindspring.com/~mgrand/pattern_synopses.htm https://refactoring.guru/design-patterns

Links

- Software Quality Assurance: http://satc.gsfc.nasa.gov/assure/agbsec3.txt
- Software Testing: http://en.wikipedia.org/wiki/Software_testing
- GUI Software Testing: http://en.wikipedia.org/wiki/GUI_software_testing
- Regression Testing: http://en.wikipedia.org/wiki/Regression_testing
- Junit Test Example: http://www.cs.unc.edu/~weiss/COMP401/s08-27-JUnitTestExample.doc
- https://www.test-institute.org/What_is_Software_Quality_Assurance.php